TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

Gestión de Desarrollos de Software

Trabajo Práctico

Gestión de Artefactos, Documentación Automatizada (Docs-as-Code) y Redacción Técnica

Objetivo

Aplicar los conceptos de clasificación y organización de artefactos de software, estándares de mantenibilidad, la metodología Docs-as-Code (pilares, diagramas en texto plano y automatización), la elaboración de manuales e instructivos (POEs) y el uso asistido de Inteligencia Artificial Generativa a un caso simplificado de desarrollo de software.

Caso

La empresa TechGuard se encuentra desarrollando un módulo de software denominado "AuthService" (API de autenticación y gestión de usuarios). El equipo anterior dejó el código desorganizado, la documentación dispersa en archivos Word/PDF desactualizados (sufriendo de documentation drift) y los diagramas de arquitectura en formato de imagen (.png) difíciles de mantener. Para resolver estos problemas y garantizar la escalabilidad y mantenibilidad del sistema, la dirección ha decidido reorganizar el proyecto bajo una estructura estándar, adoptar el enfoque Docs-as-Code e integrar herramientas de Inteligencia Artificial Generativa para acelerar la creación de manuales y diagramas.

Consignas

● 1. Organización de Artefactos y Principios de Documentación

○ Estructura de carpetas: Indique en qué carpeta estándar de la estructura de proyecto (src/, docs/, tests/, config/, assets/, README.md) debe ubicarse cada uno de los siguientes artefactos:

- ■ Archivo de variables de entorno (.env.example). config/
- ■ Imágenes de capturas de pantalla para el manual. -→ assets/
- ■ Código fuente de los controladores de autenticación. → src/
- ■ Pruebas unitarias de integración. → tests/
- ■ Especificación técnica del sistema en Markdown. →docs/

○ Calidad de la documentación: Defina brevemente las "4 C" de la buena documentación y explique cómo aplicarlas al redactar el archivo README.md principal del proyecto.

- Clara: fácil de entender. En el README: lenguaje simple, sin jerga innecesaria, con un orden lógico (qué es AuthService, cómo instalarlo, cómo usarlo).
- Concisa: sin redundancias. En el README: ir al punto, sin repetir información y enlazando a docs/ para el detalle en lugar de copiarlo.
- Correcta: exactamente precisa. En el README: los comandos de instalación, variables de entorno y endpoints deben coincidir con el código real, y probarse antes de publicar.

- Completa: con toda la información que el lector objetivo necesita. En el README: descripción general, requisitos previos, instalación, configuración (.env.example), ejemplo de uso y enlaces a la documentación técnica.

● 2. Comparativa: Documentación Tradicional vs. Docs-as-Code

Complete la siguiente tabla comparativa especificando cómo se gestiona cada aspecto del proyecto "AuthService" bajo el enfoque tradicional versus el enfoque Docs-as-Code:

Aspecto	**Documentación tradicional**	**Enfoque Docs-as-Code**
¿Dónde se escribe/almacena?	Archivos Word/PDF o wikis externas separadas	Archivos Markdown (.md) dentro de docs/ en el mismo repositorio Git de AuthService
Control de versiones	Manejo manual de versiones por nombre de archivo (doc_v2_final_FINAL.docx), sin saber quién cambió qué	Registro mediante Git (commits, historial, autor)
Revisión de cambios	Comentarios manuales o envíos por correo	Pull Request con revisión y comentarios línea por línea, aprobada por otro miembro del equipo antes del merge
Publicación	Manual: exportar, subir y compartir el enlace	Automática mediante pipeline CI/CD tras cada merge
Formato del archivo	Binario / Procesador de texto (DOCX, PDF)	Texto plano (Markdown), legible sin renderizar y comparable línea por línea con Git

● 3. Diagramación como Código (Mermaid.js) y Mantenibilidad

○ Análisis del problema: Explique qué es el documentation drift (desviación de la documentación) y por qué los diagramas guardados en formatos de imagen tradicionales (.png, .jpg) favorecen este problema.

Ocurre cuando la documentación se separa del código y queda desactualizada. Los diagramas en formato imagen (.png, .jpg) favorecen este problema porque no se pueden versionar ni comparar cambios fácilmente en Git.

○ Construcción en Mermaid: Escriba la sintaxis en Mermaid.js (orientación flowchart TD) para representar el proceso de inicio de sesión de un usuario:

- ■ Inicio -> Usuario ingresa credenciales.
- ■ ¿Credenciales válidas?
- ■ Si son válidas: Generar Token JWT -> Acceso al sistema.
- ■ Si no son válidas: Mostrar error -> Reintentar.




```
flowchart TD
    A([Inicio]) --> B[Usuario ingresa credenciales]
    B --> C{¿Credenciales válidas?}
    C -->|Sí| D[Generar Token JWT]
    D --> E([Acceso al sistema])
    C -->|No| F[Mostrar error]
    F --> B
```

○ Explique brevemente por qué guardar este diagrama como texto plano dentro del repositorio facilita la colaboración mediante Pull Requests.

Porque se puede revisar línea por línea en un Pull Request, facilitando colaboración y trazabilidad.

● 4. Manuales, Instructivos y Procedimientos Operativos Estándar (POE)

○ Tipología de manuales: Indique qué tipo de documento (Manual de Usuario, Manual Técnico, Manual de Instalación o Guía Rápida) corresponde a cada uno de los siguientes escenarios:

■ Un nuevo desarrollador se suma al equipo y necesita configurar su entorno local y la base de datos.

Manual de instalación.

■ Un administrador de sistemas necesita consultar la arquitectura de componentes, esquemas de BD y endpoints de la API.

Manual Técnico.

■ Un cliente final necesita aprender a navegar por el sistema y realizar las tareas cotidianas en lenguaje simple.

Manual de Usuario.

● 5. Redacción Generativa y Automatización con IA

Responda brevemente:

○ ¿Cómo se integra la Inteligencia Artificial Generativa dentro del pipeline de Docs-as-Code para la creación de diagramas y borradores de manuales?

La IA generativa se incorpora en la etapa de redacción, antes de que el contenido entre al flujo de Git. El desarrollador describe en lenguaje natural lo que quiere documentar (un proceso, una arquitectura, las relaciones entre clases) y aporta contexto (código, flujos, capturas). La IA genera el código Mermaid o un borrador del manual en Markdown, el desarrollador lo revisa y ajusta, y luego lo guarda en el repositorio y lo sube en un commit. A partir de ahí sigue el flujo habitual: Pull Request con revisión de otro miembro del equipo, validaciones automáticas de Integración Continua y publicación automática por Despliegue Continuo. Cuando el sistema cambia, se repite el proceso.

○ Mencione un beneficio clave y una limitación o riesgo de utilizar modelos de IA para generar documentación técnica sin supervisión humana (human-in-the-loop).

Beneficio clave: velocidad. Elimina el tiempo que se pierde recordando la sintaxis o estructurando un diagrama o manual desde cero, y permite obtener un borrador en horas en lugar de días.

Limitación: la IA puede inventar pasos que parecen lógicos pero son incorrectos cuando no tiene suficiente contexto del sistema, y puede omitir advertencias críticas (por ejemplo, pasos que causan pérdida de datos o problemas de seguridad). Por eso el resultado es un borrador que debe ser revisado por un desarrollador o especialista antes de publicarse.